DISEÑO COMPUTACIONAL · ANEXO

Catálogo de endpoints

El puente hacia la construcción: cada endpoint por incremento, con su caso de uso, la operación que lo resuelve, las entidades que toca y quién puede llamarlo.

PROYECTO

AdaptiveLearn

CURSO

DSE-2 · Diseño y Desarrollo de Software Educativo II

METODOLOGÍA

MODESEC · Fase III

FECHA

22 de octubre de 2026

Metodología: MODESEC — Fase III, Diseño Computacional. Anexo: no es una sub-etapa de MODESEC. **Fecha:** 2026-10-22. **Archivo generado** por `api/generar_api.py`; no se edita a mano.

MODESEC cierra el diseño computacional con el modelado de la base de datos; no pide describir lo que el frontend le pide al backend. Este anexo lo describe porque es lo primero que hace falta para construir (Fase IV). Cada endpoint dice qué caso de uso atiende (`12`), qué operación del diagrama de clases lo resuelve (`13`), en qué diagrama de secuencia se ve (`15`), qué entidades crea, lee, modifica o borra —las mismas de la matriz CRUD de `16` §7—, quién puede llamarlo y si ya existe en el backend del incremento I1.

Es, casi literalmente, la lista de tareas de cada incremento: cada endpoint lleva un número, de E-01 a E-35, para repartirlo.

---

1. En cifras

Incremento	Subsistemas	Endpoints	Hechos en I1	Por ajustar	Faltan	Opcionales
I1 — Acceso	SUB-01	13	8	2	2	1
I2 — Experiencia y banco	SUB-02, SUB-03, SUB-06	13	0	0	13	0
I3 — Diagnóstico	SUB-04	6	0	0	6	0
I4 — Reporte y seguimiento	SUB-05	3	0	0	3	0
Total		**35**	**8**	**2**	**24**	**1**

De los 35, 10 ya existen en el backend I1 (`backend/` del repositorio): lo dice su código, revisado el 2026-10-26. 2 de ellos funcionan, pero el diseño les pide algo distinto. Los incrementos I2, I3 e I4 están por construir.

---

2. Convenciones

2.1 Quién puede llamar

Todo token es un JWT en la cabecera `Authorization: Bearer`; su tipo va dentro. Quién puede llamar sale del actor del caso de uso: un endpoint de un caso de uso del Docente no lo llama un Estudiante, y el verificador lo comprueba.

Acceso	Quién
Público	Sin token.
Token de registro	El que entrega el registro (CU-01), o la primera entrada con Google o Microsoft (CU-02). Identifica una credencial que todavía no tiene rol: sirve para verificar el correo y para elegir el rol; no abre ninguna pantalla del producto.
Token de recuperación	El que entrega la verificación del código de recuperación. Solo sirve para fijar la contraseña nueva; no abre sesión (I1 ya lo prueba: 401).
Sesión	Docente o estudiante con sesión iniciada.
Docente	Sesión con rol docente.

Acceso	Quién
Docente dueño	Sesión del docente dueño de la experiencia; para un ítem o una inscripción, el de la experiencia a la que pertenecen.
Estudiante	Sesión con rol estudiante.
Estudiante dueño	Sesión del estudiante dueño de la inscripción.

2.2 Errores

Código	Cuándo
`400`	Los datos no cumplen una regla: formato, campo vacío, contraseña débil, ítem inválido.
`401`	Sin token, token vencido o de otro tipo; o credenciales incorrectas, con un mensaje que no dice qué falló (RF-01-07).
`403`	El token es válido, pero de otro rol o de quien no es el dueño. Así lo hace I1 («Modificar perfil ajeno → 403»).
`404`	No existe.
`409`	El estado no lo permite: correo ya registrado, rol ya elegido, banco congelado, experiencia ya publicada, pregunta ya respondida.
`429`	Demasiados intentos fallidos.
`503`	Fallaron el servicio de IA principal y el de respaldo; no se guardó nada.

Un endpoint que no es del dueño responde `403`, no `404`, como ya lo hace I1. Los mensajes de error van en español y nunca afirman certeza sobre el aprendizaje (RNF-02, RNF-04).

2.3 Nombres

Las rutas y los campos nuevos van en español, con los nombres del MER y del MR (`experiencias`, `inscripciones`, `codigo_dba`, `certeza`): un mismo dato no cambia de nombre entre la base, el backend y este catálogo. Las cuatro rutas que I1 heredó de la guía del curso en inglés — `/auth/register`, `/auth/login`, `/auth/profile`, `/users/:id` —, y sus campos `email`, `password` y `role`, se dejan como están: ya los usan el backend I1 y su colección de pruebas. Unificarlas es una decisión del equipo; si se toma, se hace en un solo cambio y se corrige aquí.

En **Toca**, las entidades son las del MER (`16`): REPORTE no es una tabla, se calcula al abrirlo (ADR-022).

---

3. I1 — Acceso

N.º	Endpoint	Casos de uso	Operación	Quién	Estado
E-01	`POST /auth/register`	CU-01	`ServicioAcceso.registrar`	Público	Hecho en I1
E-02	`POST /auth/verificar-correo`	CU-03	`ServicioAcceso.verificarCorreo`	Token de registro	Hecho en I1
E-03	`POST /auth/reenviar-codigo`	CU-03	`ServicioAcceso.reenviarCodigo`	Público	Hecho en I1

N.º	Endpoint	Casos de uso	Operación	Quién	Estado
E-04	`POST /auth/google`	CU-02	`ServicioAcceso.entrarConProveedor`	Público	Falta
E-05	`POST /auth/microsoft`	CU-02	`ServicioAcceso.entrarConProveedor`	Público	Falta
E-06	`POST /auth/elegir-rol`	CU-04 CU-05	`ServicioAcceso.elegirRol`	Token de registro	Por ajustar
E-07	`POST /auth/login`	CU-06	`ServicioAcceso.iniciarSesion`	Público	Hecho en I1
E-08	`POST /auth/recuperar-contrasena`	CU-07	`ServicioAcceso.solicitarRecuperacion`	Público	Hecho en I1
E-09	`POST /auth/recuperar-contrasena/verificar`	CU-07 CU-03	`ServicioAcceso.verificarCorreo`	Público	Hecho en I1
E-10	`POST /auth/nueva-contrasena`	CU-07	`ServicioAcceso.restablecerContrasena`	Token de recuperación	Hecho en I1
E-11	`GET /auth/profile`	CU-09	`ServicioAcceso.perfil`	Sesión	Hecho en I1
E-12	`PATCH /users/:id`	CU-09	`ServicioAcceso.editarPerfil`	Sesión	Por ajustar
E-13	`POST /auth/cerrar-sesion`	CU-08	`ServicioAcceso.cerrarSesion`	Sesión	Opcional

E-01 · `POST /auth/register`

CU-01 Registrarse con correo y contraseña. Operación: `ServicioAcceso.registrar(correo: string, contrasena: string)`; se ve en `15` §1. **Hecho en I1.**

- **Quién:** Público.
- **Recibe:** `email`, `password` (nombres de campo de I1).
- **Responde:** 201: token de registro. El código de 6 dígitos sale al correo; en desarrollo, I1 lo imprime en la terminal del servidor.
- **Toca:** CREDENCIAL (crea, lee).
- **Reglas y errores:**
  - `400` contraseña sin 8 caracteres, una mayúscula y un número (RF-01-01).
  - `409` el correo ya tiene cuenta; si es de Google o Microsoft, el mensaje dice qué método usar y no vincula (RF-01-04, RI-3).
- **Nota:** I1 acepta además `fullName`, que viene de la guía. En el diseño los nombres se piden en Completar perfil (CU-04) y se guardan al elegir el rol.

E-02 · `POST /auth/verificar-correo`

CU-03 Verificar correo con código. Operación: `ServicioAcceso.verificarCorreo(correo: string, codigo: string)`; se ve en `15` §1. **Hecho en I1.**

- **Quién:** Token de registro.

- **Recibe:** `codigo`; la credencial la identifica el token de registro.
- **Responde:** 200: correo verificado; sigue Completar perfil (CU-04).
- **Toca:** CREDENCIAL (lee, modifica).
- **Reglas y errores:**
  - `400` código incorrecto: se cuenta un intento fallido (CU-03 3a).
  - `400` código vencido: hay que pedir otro (CU-03 3b).
  - `429` demasiados intentos fallidos (`intentos_codigo`).

E-03 · `POST /auth/reenviar-codigo`

CU-03 Verificar correo con código. Operación: `ServicioAcceso.reenviarCodigo(correo: string)`. **Hecho en I1.**

- **Quién:** Público.
- **Recibe:** `email`.
- **Responde:** 201 siempre, `{ ok: true }`, exista o no la cuenta. Solo si existe, es de correo y ya pasaron 30 s desde el último envío, genera un código nuevo que reemplaza al anterior (CU-03 1a).
- **Toca:** CREDENCIAL (lee, modifica).
- **Reglas y errores:**
  - `400` el correo no tiene formato de correo.
  - Pedirlo antes de 30 s responde igual y no reenvía: un 429 delataría que la cuenta existe (RNF-07).
- **Nota:** Sirve a los dos flujos, registro y recuperación, como la ventana 015 (CU-03). Así lo hace I1.

E-04 · `POST /auth/google`

CU-02 Entrar con Google o Microsoft. Operación: `ServicioAcceso.entrarConProveedor(token: string): Sesion`. **Falta.**

- **Quién:** Público.
- **Recibe:** `token` que entregó Google.
- **Responde:** 200: sesión, si la cuenta ya tiene rol. Si es nueva, crea la credencial con el correo ya verificado y responde token de registro con nombres, apellidos y foto para prellenar el perfil (CU-02 3a).
- **Toca:** CREDENCIAL (crea, lee); DOCENTE (lee); ESTUDIANTE (lee).
- **Reglas y errores:**
  - `401` Google rechaza el token, o la persona canceló (CU-02 2a).
  - `409` el correo ya está registrado con otro método: dice cuál; no crea ni vincula (RF-01-04, CU-02 3b).

E-05 · `POST /auth/microsoft`

CU-02 Entrar con Google o Microsoft. Operación: `ServicioAcceso.entrarConProveedor(token: string): Sesion`. **Falta.**

- **Quién:** Público.
- **Recibe:** `token` que entregó Microsoft.
- **Responde:** Igual que `POST /auth/google`.

- **Toca:** CREDENCIAL (crea, lee); DOCENTE (lee); ESTUDIANTE (lee).
- **Reglas y errores:**
  - `401` Microsoft rechaza el token, o la persona canceló (CU-02 2a).
  - `409` el correo ya está registrado con otro método (RF-01-04).
- **Nota:** Sin Apple: exige cuenta de desarrollador de pago (RF-01-02).

E-06 · `POST /auth/elegir-rol`

CU-04 Completar perfil · CU-05 Elegir rol. Operación: `ServicioAcceso.elegirRol(rol: string, perfil: Perfil): Sesion`; se ve en `15` §1. **Por ajustar.**

- **Quién:** Token de registro.
- **Recibe:** `role` (docente o estudiante), `nombres`, `apellidos` y `foto` opcional, en multipart/form-data.
- **Responde:** 201: sesión del rol. En una sola operación crea el docente o el estudiante con su perfil y lo asocia a la credencial (CU-05 paso 3).
- **Toca:** CREDENCIAL (lee, modifica); DOCENTE (crea); ESTUDIANTE (crea).
- **Reglas y errores:**
  - `400` nombres o apellidos vacíos; foto que no es JPG ni PNG, o de más de 5 MB (CU-04 2a).
  - `403` el correo de la credencial no está verificado (precondición de CU-04).
  - `409` la credencial ya tiene rol: el rol no se cambia (RF-01-06, RI-12). I1 ya lo prueba.
- **Qué ajustar:** En I1 recibe solo `role`, y el perfil se completa antes con `PATCH /users/:id`. En el diseño la cuenta nace con su perfil al elegir el rol: CU-04 dice que «estos datos se guardan al elegir el rol» (RF-01-05), y nombres, apellidos y foto viven en DOCENTE o ESTUDIANTE, que no existen antes del rol.

E-07 · `POST /auth/login`

CU-06 Iniciar sesión. Operación: `ServicioAcceso.iniciarSesion(correo: string, contrasena: string): Sesion`; se ve en `15` §2. **Hecho en I1.**

- **Quién:** Público.
- **Recibe:** `email`, `password`.
- **Responde:** 201 (el de NestJS por defecto, como en I1): token de sesión y rol. Si la credencial todavía no tiene rol, token de registro para seguir en Completar perfil (CU-06 3a, decisión por confirmar).
- **Toca:** CREDENCIAL (lee, modifica); DOCENTE (lee); ESTUDIANTE (lee).
- **Reglas y errores:**
  - `401` correo o contraseña incorrectos, sin decir cuál (RF-01-07); se cuenta un intento fallido (`intentos_login`).
  - `409` la cuenta es de Google o Microsoft: dice qué método usar (RF-01-04, CU-06 2c).
  - `429` entrada bloqueada por intentos fallidos (`bloqueado_hasta`).

E-08 · `POST /auth/recuperar-contrasena`

CU-07 Recuperar contraseña. Operación: `ServicioAcceso.solicitarRecuperacion(correo: string)`. **Hecho en I1.**

- **Quién:** Público.

- **Recibe:** `email`.
- **Responde:** 202 siempre, exista o no la cuenta; si existe, envía el código (RNF-07). I1 ya responde igual en los dos casos.
- **Toca:** CREDENCIAL (lee, modifica).
- **Reglas y errores:**
  - `400` el correo no tiene formato de correo (CU-07 paso 2).
  - Pedirlo otra vez antes de 30 s responde igual, 202, y no reenvía: un 429 delataría que la cuenta existe (RNF-07).

E-09 · `POST /auth/recuperar-contrasena/verificar`

CU-07 Recuperar contraseña · CU-03 Verificar correo con código. Operación: `ServicioAcceso.verificarCorreo(correo: string, codigo: string)`. **Hecho en I1.**

- **Quién:** Público.
- **Recibe:** `email`, `codigo`.
- **Responde:** 200: token de recuperación.
- **Toca:** CREDENCIAL (lee, modifica).
- **Reglas y errores:**
  - `400` código incorrecto o vencido (CU-03 3a, 3b).
  - `429` demasiados intentos fallidos.

E-10 · `POST /auth/nueva-contrasena`

CU-07 Recuperar contraseña. Operación: `ServicioAcceso.restablecerContrasena(correo: string, codigo: string, nueva: string)`. **Hecho en I1.**

- **Quién:** Token de recuperación.
- **Recibe:** `password` nueva, con el token de recuperación.
- **Responde:** 204: contraseña cambiada; la pantalla lleva a iniciar sesión (CU-07 paso 5).
- **Toca:** CREDENCIAL (lee, modifica).
- **Reglas y errores:**
  - `400` la contraseña no cumple las reglas del registro (RF-01-01).
  - `401` token de recuperación inválido o vencido; un token de sesión no sirve aquí.
- **Nota:** `restablecerContrasena(correo, codigo, nueva)` va partida en dos peticiones, verificar el código y fijar la contraseña, con el token de recuperación en medio. Así lo hace I1, y es compatible con el diseño.

E-11 · `GET /auth/profile`

CU-09 Editar mi perfil. Operación: `ServicioAcceso.perfil(): Perfil`. **Hecho en I1.**

- **Quién:** Sesión.
- **Recibe:** nada.
- **Responde:** 200: correo, rol, nombres, apellidos y foto: lo que muestra el bloque de perfil del menú lateral (CU-09 paso 1).
- **Toca:** CREDENCIAL (lee); DOCENTE (lee); ESTUDIANTE (lee).

- **Reglas y errores:**
  - `401` sin token, o con uno de registro o de recuperación.

E-12 · `PATCH /users/:id`

CU-09 Editar mi perfil. Operación: `ServicioAcceso.editarPerfil(perfil: Perfil)`. **Por ajustar.**

- **Quién:** Sesión.
- **Recibe:** `nombres`, `apellidos` y `foto` opcional, en multipart/form-data.
- **Responde:** 200: el perfil guardado.
- **Toca:** DOCENTE (lee, modifica); ESTUDIANTE (lee, modifica).
- **Reglas y errores:**
  - `400` nombres o apellidos vacíos, o foto inválida: la foto no cambia (CU-09 3a).
  - `400` intenta cambiar el correo o el rol, que no se editan (CU-09 paso 2).
  - `403` el perfil no es el propio. I1 ya lo prueba.
- **Qué ajustar:** En I1 sirve para completar el perfil **antes** de elegir el rol. En el diseño eso pasa en `POST /auth/elegir-rol`, y esta ruta queda solo para Editar mi perfil (CU-09), sobre la fila de DOCENTE o ESTUDIANTE; además recibe la foto.

E-13 · `POST /auth/cerrar-sesion`

CU-08 Cerrar sesión. Operación: `ServicioAcceso.cerrarSesion()`. **Opcional.**

- **Quién:** Sesión.
- **Recibe:** nada.
- **Responde:** 204.
- **Toca:** nada guardado.
- **Por qué es opcional:** Con JWT basta con que el navegador borre el token: cerrar sesión no cambia nada guardado (CU-08). Hace falta solo si se quiere invalidar tokens en el servidor.

---

4. I2 — Experiencia y banco

4.1 SUB-06 · Catálogo curricular

N.º	Endpoint	Casos de uso	Operación	Quién	Estado
E-14	`GET /catalogo/dbas`	CU-31	`Catalogo.dbas`	Docente	Falta
E-15	`GET /catalogo/dbas/:codigo`	CU-31 CU-11	`Catalogo.dba`	Docente	Falta

E-14 · `GET /catalogo/dbas`

CU-31 Consultar catálogo curricular. Operación: `Catalogo.dbas(codigo_area: string, numero_grado: int, texto: string): DBA [*]`; se ve en `15` §3. **Falta.**

- **Quién:** Docente.
- **Recibe:** `codigo_area` (`MAT`, `LEN`, `CN` o `CS`), `numero_grado` (1 a 11) y `q` (texto), en la consulta.

- **Responde:** 200: los DBA del área, cada uno con su código `ÁREA-GG-NN`, su grado (número y nombre) y su enunciado.
- **Toca:** DBA (lee); AREA (lee); GRADO (lee).
- **Reglas y errores:**
  - `400` falta el área, o no es una de las 4 (CU-11 paso 1); o el grado no está entre 1 y 11.
- **Nota:** Las 4 áreas son fijas (CU-11 paso 1, ADR-033): la pantalla las tiene, con los mismos códigos de la tabla `area` (ADR-036), y no se piden al backend.

E-15 · `GET /catalogo/dbas/:codigo`

CU-31 Consultar catálogo curricular · CU-11 Crear experiencia. Operación: `Catalogo.dba(codigo_dba: string): DBA`. **Falta.**

- **Quién:** Docente.
- **Recibe:** nada (`:codigo` es el código `ÁREA-GG-NN`, ej. `MAT-08-01`).
- **Responde:** 200: enunciado oficial, ejemplo, área, grado, el documento oficial del que se copió (título y versión) y las evidencias en el orden oficial, cada una marcada si admite o no formato cerrado (RF-06-02). El enunciado prellena el nombre en el Paso 3.
- **Toca:** DBA (lee); AREA (lee); GRADO (lee); DOCUMENTO_OFICIAL (lee); EVIDENCIA (lee); EVIDENCIA_DICTAMEN (lee).
- **Reglas y errores:**
  - `404` no existe ese DBA.

4.2 SUB-02 · Gestión de experiencias

N.º	Endpoint	Casos de uso	Operación	Quién	Estado
E-16	`GET /experiencias`	CU-10	`ServicioExperiencias.listar`	Docente	Falta
E-17	`POST /experiencias`	CU-11 CU-16	`ServicioExperiencias.crear`	Docente	Falta
E-18	`GET /experiencias/:id`	CU-12 CU-13 CU-15	`ServicioExperiencias.consultar`	Docente dueño	Falta
E-19	`POST /experiencias/:id/publicar`	CU-13	`ServicioExperiencias.publicar`	Docente dueño	Falta
E-20	`GET /experiencias/:id/codigo`	CU-14	`ServicioExperiencias.codigoInvitacion`	Docente dueño	Falta
E-21	`DELETE /experiencias/:id`	CU-15	`ServicioExperiencias.eliminar`	Docente dueño	Falta

E-16 · `GET /experiencias`

CU-10 Consultar mis experiencias. Operación: `ServicioExperiencias.listar(docente: Docente): Experiencia [*]`. **Falta.**

- **Quién:** Docente.
- **Recibe:** `estado` (borrador o publicada) y `q`, en la consulta.

- **Responde:** 200: solo las experiencias del docente, cada una con estado, nombre, área, grado y número de estudiantes (CU-10 paso 1).
- **Toca:** EXPERIENCIA (lee); DBA (lee); AREA (lee); GRADO (lee); INSCRIPCION (lee).

E-17 · `POST /experiencias`

CU-11 Crear experiencia · CU-16 Generar banco con IA. Operación: `ServicioExperiencias.crear(docente: Docente, codigo_dba: string, nombre: string): Experiencia`; se ve en `15` §3. **Falta.**

- **Quién:** Docente.
- **Recibe:** `codigo_dba`, `nombre`.
- **Responde:** 201: la experiencia en Borrador, con 12 ítems por evidencia incluida, su código de invitación, y el servicio y el modelo de IA que generaron el banco.
- **Toca:** EXPERIENCIA (crea); EXPERIENCIA_EVIDENCIA (crea); ITEM (crea); OPCION (crea); DBA (lee); EVIDENCIA (lee); EVIDENCIA_DICTAMEN (lee).
- **Reglas y errores:**
  - `503` fallaron el servicio principal y el de respaldo: no se guarda nada, y la pantalla conserva nombre y DBA para reintentar (RF-02-03, ADR-027, CU-11 5a).
  - Si el principal falla o pasa del tiempo límite (RNF-13), se pide el banco entero al de respaldo; un banco no mezcla servicios (RF-03-02, RI-17).
  - Solo entran las evidencias cuyo dictamen admite formato cerrado (RI-5), y cada ítem sale con el formato de ese dictamen (RI-19).
  - `404` no existe el DBA.
- **Nota:** Puede tardar minutos: ver la generación asíncrona en «Adicionales que conviene agregar».

E-18 · `GET /experiencias/:id`

CU-12 Consultar experiencia · CU-13 Publicar experiencia · CU-15 Eliminar experiencia. Operación: `ServicioExperiencias.consultar(id: int): Experiencia`; se ve en `15` §4 y §8. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** nada.
- **Responde:** 200: encabezado (nombre, estado, fecha, área, grado, número de inscritos), el DBA, cuántos ítems activos faltan por evidencia para publicar (CU-13 1a), y cuántos inscritos y respuestas se perderían al eliminarla (CU-15 paso 2).
- **Toca:** EXPERIENCIA (lee); DBA (lee); AREA (lee); GRADO (lee); EVIDENCIA (lee); EXPERIENCIA_EVIDENCIA (lee); ITEM (lee); INSCRIPCION (lee); RESPUESTA (lee).
- **Reglas y errores:**
  - `403` la experiencia no es del docente.
  - `404` no existe.

E-19 · `POST /experiencias/:id/publicar`

CU-13 Publicar experiencia. Operación: `ServicioExperiencias.publicar(id: int)`; se ve en `15` §4. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** nada.

- **Responde:** 200: la experiencia Publicada, con su fecha de publicación; el banco queda congelado (RF-03-08).
- **Toca:** EXPERIENCIA (lee, modifica); EXPERIENCIA_EVIDENCIA (lee); ITEM (lee).
- **Reglas y errores:**
  - `409` ya está Publicada: publicar no se deshace (ADR-026).
  - `409` alguna evidencia tiene menos de 6 ítems activos; dice cuántos faltan (RF-02-05, RI-14).

E-20 · `GET /experiencias/:id/codigo`

CU-14 Compartir código de invitación. Operación: `ServicioExperiencias.codigoInvitacion(id: int): string`. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** nada.
- **Responde:** 200: el código de 6 caracteres, sin 0/O ni 1/I/L (RI-3).
- **Toca:** EXPERIENCIA (lee).
- **Reglas y errores:**
  - `409` la experiencia está en Borrador: el código existe, pero no se muestra (RF-02-06, CU-14 1a).

E-21 · `DELETE /experiencias/:id`

CU-15 Eliminar experiencia. Operación: `ServicioExperiencias.eliminar(id: int)`; se ve en `15` §8. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** nada.
- **Responde:** 204.
- **Toca:** EXPERIENCIA (lee, borra).
- **Borra en cascada** (de las llaves foráneas de `17`): EXPERIENCIA_EVIDENCIA, INSCRIPCION, ITEM, OPCION, PRESENTACION, RESPUESTA, RESPUESTA_OPCION, RESULTADO.
- **Reglas y errores:**
  - Irreversible; las cuentas de los estudiantes siguen existiendo (CU-15).

4.3 SUB-03 · Banco de preguntas

N.º	Endpoint	Casos de uso	Operación	Quién	Estado
E-22	`GET /experiencias/:id/banco`	CU-17	`ServicioBanco.banco`	Docente dueño	Falta
E-23	`PATCH /items/:id`	CU-18	`ServicioBanco.editarItem`	Docente dueño	Falta
E-24	`POST /items/:id/descartar`	CU-19	`ServicioBanco.descartar`	Docente dueño	Falta
E-25	`POST /items/:id/reincluir`	CU-19	`ServicioBanco.reincluir`	Docente dueño	Falta
E-26	`POST /experiencias/:id/evidencias/:n/items`	CU-20	`ServicioBanco.agregarItem`	Docente dueño	Falta

E-22 · `GET /experiencias/:id/banco`

CU-17 Revisar banco. Operación: `ServicioBanco.banco(id_experiencia: int): ExperienciaEvidencia [*]`. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** `formato` y `q`, en la consulta.
- **Responde:** 200: las evidencias en el orden oficial del DBA, cada una con su formato, sus ítems —opciones y respuesta correcta— y cuántos activos le faltan para publicar (RF-03-04).
- **Toca:** EXPERIENCIA (lee); EXPERIENCIA_EVIDENCIA (lee); ITEM (lee); OPCION (lee).
- **Reglas y errores:**
  - Con la experiencia Publicada responde igual, marcada de solo lectura (CU-17 4a).

E-23 · `PATCH /items/:id`

CU-18 Editar ítem. Operación: `ServicioBanco.editarItem(id_item: int, enunciado: string, opciones: Opcion [*])`. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** `enunciado` y la lista completa de `opciones` (`texto`, `es_correcta`, `orden`, `grupo`).
- **Responde:** 200: el ítem guardado; si venía de la IA, su origen pasa a `editado_docente` (RF-03-03).
- **Toca:** ITEM (lee, modifica); OPCION (crea, lee, modifica, borra).
- **Reglas y errores:**
  - `400` ítem inválido: algo vacío, menos opciones que el mínimo del formato, dos correctas en selección única, orden incompleto o repetido (CU-18 paso 3, RI-4).
  - `409` la experiencia está Publicada: el banco es de solo lectura (RF-03-08, ADR-026).

E-24 · `POST /items/:id/descartar`

CU-19 Descartar o reincluir ítem. Operación: `ServicioBanco.descartar(id_item: int)`. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** nada.
- **Responde:** 204: el ítem queda excluido, sin borrarse y sin confirmación (RF-03-06).
- **Toca:** ITEM (modifica).
- **Reglas y errores:**
  - `409` la experiencia está Publicada: el banco es de solo lectura (RF-03-08, ADR-026).

E-25 · `POST /items/:id/reincluir`

CU-19 Descartar o reincluir ítem. Operación: `ServicioBanco.reincluir(id_item: int)`. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** nada.
- **Responde:** 204: el ítem vuelve a estar activo, con el mismo contenido.
- **Toca:** ITEM (modifica).
- **Reglas y errores:**
  - `409` la experiencia está Publicada: el banco es de solo lectura (RF-03-08, ADR-026).

E-26 · `POST /experiencias/:id/evidencias/:n/items`

CU-20 Agregar pregunta propia. Operación: `ServicioBanco.agregarItem(id_experiencia: int, numero_evidencia: int): Item`. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** `:n` es el número de la evidencia; el cuerpo, como en `PATCH /items/:id`.
- **Responde:** 201: ítem activo, de origen `creado_docente`, con el formato del dictamen de esa evidencia (RI-19).
- **Toca:** ITEM (crea); OPCION (crea); EVIDENCIA_DICTAMEN (lee).
- **Reglas y errores:**
  - `400` ítem inválido, con las reglas de CU-18.
  - `404` esa evidencia no está incluida en la experiencia (RI-5, RI-18).
  - `409` la experiencia está Publicada: el banco es de solo lectura (RF-03-08, ADR-026).
- **Nota:** Se llama al guardar, no al abrir el formulario: la pantalla arma el ítem vacío con el formato que ya trae el banco. Así «Cancelar» no deja nada (CU-20 3a).

---

5. I3 — Diagnóstico

N.º	Endpoint	Casos de uso	Operación	Quién	Estado
E-27	`GET /inscripciones`	CU-21	`ServicioDiagnostico.misInscripciones`	Estudiante	Falta
E-28	`POST /inscripciones`	CU-22	`ServicioDiagnostico.unirse`	Estudiante	Falta
E-29	`GET /inscripciones/:id`	CU-23 CU-26	`ServicioDiagnostico.consultar`	Estudiante dueño	Falta
E-30	`POST /inscripciones/:id/presentaciones`	CU-23	`ServicioDiagnostico.siguientePregunta`	Estudiante dueño	Falta
E-31	`POST /inscripciones/:id/respuestas`	CU-24 CU-25	`ServicioDiagnostico.responder`	Estudiante dueño	Falta
E-32	`DELETE /inscripciones/:id`	CU-26	`ServicioDiagnostico.salir`	Estudiante dueño	Falta

E-27 · `GET /inscripciones`

CU-21 Consultar mis experiencias (estudiante). Operación: `ServicioDiagnostico.misInscripciones(estudiante: Estudiante): Inscripcion [*]`. **Falta.**

- **Quién:** Estudiante.
- **Recibe:** `progreso` (no_iniciada, en_curso o completada) y `q`, en la consulta.
- **Responde:** 200: las experiencias del estudiante, cada una con su progreso, área y nombre del docente (CU-21 paso 1).
- **Toca:** INSCRIPCION (lee); EXPERIENCIA (lee); DOCENTE (lee); PRESENTACION (lee); RESULTADO (lee).
- **Reglas y errores:**

- El progreso se calcula, no se guarda: `progreso` es un atributo calculado del MER.

E-28 · `POST /inscripciones`

CU-22 Unirse con código. Operación: `ServicioDiagnostico.unirse(estudiante: Estudiante, codigo: string): Inscripcion`; se ve en `15` §5. **Falta.**

- **Quién:** Estudiante.
- **Recibe:** `codigo`, sin distinguir mayúsculas.
- **Responde:** 201: la inscripción nueva; 200 con la que ya existía si el estudiante ya estaba inscrito, sin duplicarla (RI-1, CU-22 3a).
- **Toca:** EXPERIENCIA (lee); INSCRIPCION (crea, lee).
- **Reglas y errores:**
  - `404` no hay una experiencia Publicada con ese código: el mismo mensaje para un código que no existe y para uno en Borrador (CU-22 2a, RI-7).

E-29 · `GET /inscripciones/:id`

CU-23 Tomar el diagnóstico · CU-26 Salir de la experiencia. Operación: `ServicioDiagnostico.consultar(id_inscripcion: int): Inscripcion`. **Falta.**

- **Quién:** Estudiante dueño.
- **Recibe:** nada.
- **Responde:** 200: la experiencia y su progreso, que decide «Comenzar» o «Continuar» (CU-23 paso 1), y cuántas respuestas se perderían al salir (CU-26 paso 2).
- **Toca:** INSCRIPCION (lee); EXPERIENCIA_EVIDENCIA (lee); PRESENTACION (lee); RESULTADO (lee); RESPUESTA (lee).
- **Reglas y errores:**
  - `403` la inscripción no es del estudiante.
  - No incluye resultados ni aciertos (RF-04-08, ADR-028).

E-30 · `POST /inscripciones/:id/presentaciones`

CU-23 Tomar el diagnóstico. Operación: `ServicioDiagnostico.siguientePregunta(id_inscripcion: int): Presentacion [0..1]`; se ve en `15` §6. **Falta.**

- **Quién:** Estudiante dueño.
- **Recibe:** nada.
- **Responde:** 201: el ítem presentado —enunciado, formato y opciones—. 200 con la misma pregunta si quedó una sin responder (ADR-031). 200 sin pregunta si ya concluyeron todas las evidencias, y la pantalla muestra el cierre (RF-04-11).
- **Toca:** INSCRIPCION (lee); EXPERIENCIA_EVIDENCIA (lee); ITEM (lee); OPCION (lee); PRESENTACION (crea, lee); RESULTADO (lee).
- **Reglas y errores:**
  - POST y no GET: cada llamada puede crear una PRESENTACIÓN, con su orden y su fecha (RI-8).
  - La evidencia es la primera sin concluir en el orden oficial; el ítem, uno activo no visto, al azar (ADR-024, RI-9).

- Nunca envía `es_correcta` ni `hipotesis_error` de las opciones, ni el número de pregunta, ni cuántas faltan (RF-04-08).
- `409` la experiencia ya está completada (CU-23 1a).

E-31 · `POST /inscripciones/:id/respuestas`

CU-24 Responder ítem · CU-25 Aplicar la regla adaptativa. Operación: `ServicioDiagnostico.responder(id_inscripcion: int, id_item: int, seleccion: RespuestaOpcion [*], certeza: int)`; se ve en `15` §6. **Falta.**

- **Quién:** Estudiante dueño.
- **Recibe:** `id_item`, las `opciones` elegidas (con `valor_dado` al emparejar y al ordenar) y `certeza` de 0 a 5.
- **Responde:** 201, sin decir si acertó ni cómo va (RF-04-08).
- **Toca:** PRESENTACION (lee); ITEM (lee); OPCION (lee); RESPUESTA (crea, lee); RESPUESTA_OPCION (crea); RESULTADO (crea).
- **Reglas y errores:**
  - `400` certeza fuera de 0 a 5; ninguna opción, o más de una en selección única (RI-6).
  - `400` una opción de otro ítem: el motor también la rechaza (RI-6, ciclo C4).
  - `409` el ítem no es la pregunta pendiente, o ya se respondió: una respuesta no se cambia (RNF-10).
  - Aplica la regla adaptativa (CU-25: mínimo 4, tope 6, señal fuerte con certeza de 3 a 5) y, si concluye, guarda el resultado de la evidencia como fotografía (RNF-11).

E-32 · `DELETE /inscripciones/:id`

CU-26 Salir de la experiencia. Operación: `ServicioDiagnostico.salir(id_inscripcion: int)`. **Falta.**

- **Quién:** Estudiante dueño.
- **Recibe:** nada.
- **Responde:** 204.
- **Toca:** INSCRIPCION (borra).
- **Borra en cascada** (de las llaves foráneas de `17`): PRESENTACION, RESPUESTA, RESPUESTA_OPCION, RESULTADO.
- **Reglas y errores:**
  - Para volver necesita el código otra vez, y la experiencia aparece «No iniciada» (CU-26).

---

6. I4 — Reporte y seguimiento

N.º	Endpoint	Casos de uso	Operación	Quién	Estado
E-33	`GET /experiencias/:id/reporte`	CU-27 CU-30	`ServicioReporte.reporte`	Docente dueño	Falta
E-34	`GET /experiencias/:id/estudiantes`	CU-28	`ServicioReporte.estudiantesPorEstado`	Docente dueño	Falta

N.º	Endpoint	Casos de uso	Operación	Quién	Estado
E-35	`GET /inscripciones/:id/historial`	CU-29	`ServicioReporte.historial`	Docente dueño	Falta

E-33 · `GET /experiencias/:id/reporte`

CU-27 Consultar el reporte · CU-30 Calcular estado general. Operación: `ServicioReporte.reporte(id_experiencia: int): Reporte`; se ve en `15` §7. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** nada.
- **Responde:** 200: cuántos estudiantes hay en cada estado general y, por evidencia, la distribución de resultados y su clasificación (CU-27 paso 4).
- **Toca:** REPORTE (lee); RESULTADO (lee); INSCRIPCION (lee); EXPERIENCIA_EVIDENCIA (lee).
- **Reglas y errores:**
  - Se calcula al abrirlo, no se guarda (ADR-022).
  - Ningún texto afirma certeza (RNF-04).

E-34 · `GET /experiencias/:id/estudiantes`

CU-28 Consultar estudiantes por estado. Operación: `ServicioReporte.estudiantesPorEstado(id_experiencia: int): Estudiante [*]`. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** `estado` y `q` (nombre), en la consulta.
- **Responde:** 200: los estudiantes agrupados por estado general —No muestra, No concluyente, Sin concluir, Muestra—, en orden alfabético y con su resultado en cada evidencia (CU-28).
- **Toca:** REPORTE (lee); ESTUDIANTE (lee); INSCRIPCION (lee); RESULTADO (lee).
- **Reglas y errores:**
  - Sin el correo del estudiante (ADR-029, RNF-05).

E-35 · `GET /inscripciones/:id/historial`

CU-29 Consultar historial de un estudiante. Operación: `ServicioReporte.historial(id_inscripcion: int): Presentacion [*]`. **Falta.**

- **Quién:** Docente dueño.
- **Recibe:** nada.
- **Responde:** 200: por evidencia y con su resultado, los ítems presentados en orden: qué respondió, si fue correcto y con qué certeza; las preguntas sin respuesta aparecen como tales (CU-29).
- **Toca:** PRESENTACION (lee); RESPUESTA (lee); RESPUESTA_OPCION (lee); ITEM (lee); OPCION (lee); RESULTADO (lee).
- **Reglas y errores:**
  - `403` solo el docente dueño de la experiencia de esa inscripción; el estudiante nunca (ADR-028).

7. En el backend I1, pero no del producto

El backend I1 trae también rutas que vienen de la guía del curso (JWT). AdaptiveLearn no tiene rol de administrador (`10` §2.2), así que no son del producto: se quitan antes de entregarlo, o se dejan solo en desarrollo.

Ruta	Qué es
`POST /users`	Crear usuario
`GET /users`	Listar usuarios
`GET /users/:id`	Ver un usuario por su id
`GET /example/protected`	Ruta protegida de ejemplo
`GET /example/docente`	Ruta de ejemplo con rol docente

8. Adicionales que conviene agregar

No los pide ningún requerimiento; ayudan a construir, a probar o a presentar. Ninguno cambia lo que el sistema hace.

Qué	Para qué	Efecto en el MER
`GET /docs`	Documentación interactiva (Swagger/OpenAPI, `@nestjs/swagger`): el docente puede probar cada endpoint desde el navegador, sin Insomnia.	Ninguno.
`GET /health`	Comprobar en la demostración que el servidor y la base responden.	Ninguno.
`POST /auth/refresh`	Renovar la sesión sin volver a pedir la contraseña.	Si los tokens de renovación se guardan para poder revocarlos, es una tabla que el MER no tiene.
Generación asíncrona: `POST /experiencias` responde 202 y la espera consulta `GET /experiencias/generaciones/:id`	Generar el banco puede tardar minutos (RNF-13), y una sola petición larga puede cortarse. La pantalla de espera (RNF-12) consulta hasta que termine. El «todo o nada» (ADR-027) no cambia: la experiencia se guarda solo al final.	Si el estado de la generación se guarda en la base, es una tabla nueva, fuera del MER; en memoria, ninguno. Decidirlo al medir RNF-13 en I2.
Límite de peticiones por IP en login, códigos y recuperación (no es un endpoint: `@nestjs/throttler`)	Complementa el bloqueo por credencial que ya tiene I1.	Ninguno.

9. Lo que no se debe agregar

Contradice una decisión ya tomada. El verificador comprueba que ninguna ruta del catálogo lo haga.

No agregar	Por qué
Regenerar el banco, o pedirle a la IA más ítems	ADR-026 (`10` §2.4)

No agregar	Por qué
Despublicar, o editar el banco de una experiencia publicada	ADR-026, RF-03-08 (`10` §2.4)
Que el docente elija qué evidencias entran, o agregue evidencias propias	QA-010, QA-001, RI-5 (`10` §2.4)
Mostrarle al estudiante sus resultados, su historial o el reporte	ADR-028, RF-04-08 (`10` §2.4)
Cambiar el rol	RF-01-06, RI-12
Borrar un ítem: descartar no borra	CU-19, RF-03-06
Modificar o borrar una presentación, una respuesta o un resultado	RNF-10, RNF-11, ADR-021
Administrar usuarios: listar, crear o borrar cuentas	no hay rol administrador (`10` §2.2)

---

10. Operaciones de servicio sin ruta propia

Toda operación de una clase «servicio» del diagrama de clases tiene su endpoint, salvo estas:

- `GeneradorBanco.generar()`: la llama `ServicioExperiencias.crear`: CU-11 incluye CU-16, y la generación es parte de crear la experiencia (todo o nada, ADR-027).
- `ReglaAdaptativa.concluir()`: la llama `ServicioDiagnostico.responder`: CU-24 incluye CU-25, y la regla se aplica a cada respuesta.

---

11. Decisiones de este catálogo

- **La foto viaja con el perfil**, en multipart/form-data, en `POST /auth/elegir-rol` y en `PATCH /users/:id`. CU-04 dice que el perfil se guarda al elegir el rol, y una ruta aparte para subir la foto no tendría ni operación ni caso de uso propios.
- **Las 4 áreas no tienen endpoint:** son fijas (CU-11 paso 1).
- **La siguiente pregunta es `POST`, no `GET`:** cada llamada puede crear una presentación. Un `GET` que escribe se repetiría solo con cualquier recarga o caché.
- **El diagrama de clases ganó tres operaciones** que la IGU necesita y no estaban: `ServicioAcceso.perfil()` (CU-09), `ServicioDiagnostico.misInscripciones()` (CU-21) y `ServicioDiagnostico.consultar()` (CU-23, CU-26). `ServicioExperiencias.consultar()` atiende también CU-13 y CU-15, como ya lo mostraban las secuencias 4 y 8 (`13`, `15`).
- **La matriz CRUD** registra que CU-23 también lee OPCION: presentar un ítem muestra sus opciones (`16` §7).

---

12. Cómo se comprueba

`python api/_verificar_api.py` comprueba:

- **Trazabilidad:** todo caso de uso tiene al menos un endpoint. La operación de cada uno existe en una clase «servicio» y atiende uno de sus casos de uso. Toda operación de servicio tiene ruta, o está en la sección anterior. Toda llamada de la IGU en los diagramas de secuencia es un endpoint de aquí.

- **Datos:** lo que toca cada endpoint sale de la matriz CRUD de sus casos de uso, y todo lo que un caso de uso hace lo hace alguno de sus endpoints. `GET` solo lee, `DELETE` borra y `PATCH` modifica. Ningún endpoint modifica un hecho (RNF-10, RNF-11) ni borra lo que ningún flujo borra.
- **Acceso e incremento:** quien llama es el actor del caso de uso, y cada endpoint cae en un solo incremento. Solo I1 tiene endpoints hechos.
- **Rutas:** ninguna repetida, ninguna de la sección «Lo que no se debe agregar», y las nuevas en español.

La secuencia en que se ve cada endpoint y lo que borra cada `DELETE` no se escriben a mano: se calculan de `uml/datos_secuencias.py` y de las llaves foráneas del modelo relacional.